So this shall wrap up our book and author routes, and now we have a working REST application. Now you can try executing CRUD on the author and book routes.

User Authentication

Once we have all our routes in place, we need to add in user authentication to make sure only logged-in users can access certain routes, so now we'll add in user login and signup routes, but before that we need to add user schema.

Create users.py inside models. In the schema we'll add two static methods to encrypt the password and verify password, and for the same we'll need a Python library called passlib, so before we create the schema, let's install passlib using PIP.

```
(venv)$ pip install passlib
```

Once done add the following code to add user schema and the methods.

```
from api.utils.database import db
from passlib.hash import pbkdf2_sha256 as sha256
from marshmallow_sqlalchemy import ModelSchema
from marshmallow import fields

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key = True)
    username = db.Column(db.String(120), unique = True,
    nullable = False)
    password = db.Column(db.String(120), nullable = False)

    def create(self):
        db.session.add(self)
```

```
        db.session.commit()
        return self

    @classmethod
    def find_by_username(cls, username):
        return cls.query.filter_by(username = username).first()

    @staticmethod
    def generate_hash(password):
        return sha256.hash(password)

    @staticmethod
    def verify_hash(password, hash):
        return sha256.verify(password, hash)

class UserSchema(ModelSchema):
    class Meta(ModelSchema.Meta):
        model = User
        sqla_session = db.session

    id = fields.Number(dump_only=True)
    username = fields.String(required=True)
```

So here we have added a class method to find a user by username, and create a user and then two static methods to generate the hash and verify it. We'll use these methods when we create the user routes.

Next create users.py in routes directory, and this is where we'll add our user login and signup routes.

For user authentication across the application, we'll use JWT (JSON Web Tokens) authentication. JWT is an open standard which defines a compact and self-contained way of securely transmitting information as a JSON object. JWT is a popular way of user authorization in the REST world. In Flask there is an open source extension called Flask-JWT-Extended which provides JWT support and other helpful methods.

Let's go ahead and install Flask-JWT-Extended.

```
(venv)$ pip install flask-jwt-extended
```

Next we'll initialize JWT module in the app in main.py so import the library in main.py.

```
from flask_jwt_extended import JWTManager
```

Next initialize JWTManager with the following code right above db.init_app().

```
jwt = JWTManager(app)
```

Once installed and initialized, let's import the needed modules for our user routes file.

```
from flask import Blueprint, request
from api.utils.responses import response_with
from api.utils import responses as resp
from api.models.users import User, UserSchema
from api.utils.database import db
from flask_jwt_extended import create_access_token
```

These are the modules we'll need for the user routes; next we'll configure the route using Blueprint with the following code.

```
user_routes = Blueprint("user_routes", __name__)
```

Next, we'll import and register the /users routes in our main.py file, so add the following code in main.py to import the user routes.

```
from api.routes.users import user_routes
```

And now right below where we have declared the other routes, add the following line of code.

```
app.register_blueprint(user_routes, url_prefix='/api/users')
```

Next, we'll create our POST user route to create a new user and add the following code in users.py inside routes.

```
@user_routes.route('/', methods=['POST'])
def create_user():
    try:
        data = request.get_json()
        data['password'] = User.generate_hash(data['password'])
        user_schmea = UserSchema()
        user, error = user_schmea.load(data)
        result = user_schmea.dump(user.create()).data
        return response_with(resp.SUCCESS_201)
    except Exception as e:
        print e
        return response_with(resp.INVALID_INPUT_422)
```

Here we are taking the user request data in a variable and then executing the generate_hash() function on the password and creating the user. Once done we'll return a 201 response.

Next we'll create a login route for the signed up users to login. Add the following code for the same.

```
@user_routes.route('/login', methods=['POST'])
def authenticate_user():
      try:
        data = request.get_json()
        current_user = User.find_by_username(data['username'])
        if not current_user:
            return response_with(resp.SERVER_ERROR_404)
        if User.verify_hash(data['password'], current_user.
        password):
            access_token = create_access_token(identity =
            data['username'])
```

```
            return response_with(resp.SUCCESS_201,
            value={'message': 'Logged in as {}'.format(current_
            user.username), "access_token": access_token})
        else:
            return response_with(resp.UNAUTHORIZED_401)
    except Exception as e:
        print e
        return response_with(resp.INVALID_INPUT_422)
```

The following code will take the username and password from request data and check if the user with the provided username exists using the find_by_username() method we created in the schema. Next if the user doesn't exist, we'll respond with 404, or else verify the password using verify_hash() function in the schema. If the user exists, we'll generate a JWT Token and respond with 200; otherwise respond with 401. So now we have our user login in place. Next we need to add jwt required decorator to the routes we want to protect. So navigate to authors.py in routes and import the decorator using the following code.

```
from flask_jwt_extended import jwt_required
```

And then before the endpoint definition, add the decorator using the following code.

```
@jwt_required
```

We'll add the decorator to the DELETE, PUT, POST, and PATCH endpoints of authors.py and books.py, and the functions should now look like this.

```
@author_routes.route('/', methods=['POST'])
@jwt_required
def create_author():
      ....Function code
```

Let's go ahead and test our user endpoints. Open Postman and request the POST users endpoint with a username and password. We'll use the following sample data.

```
{
        "username" : "admin",
        "password" : "flask2019"
 }
```

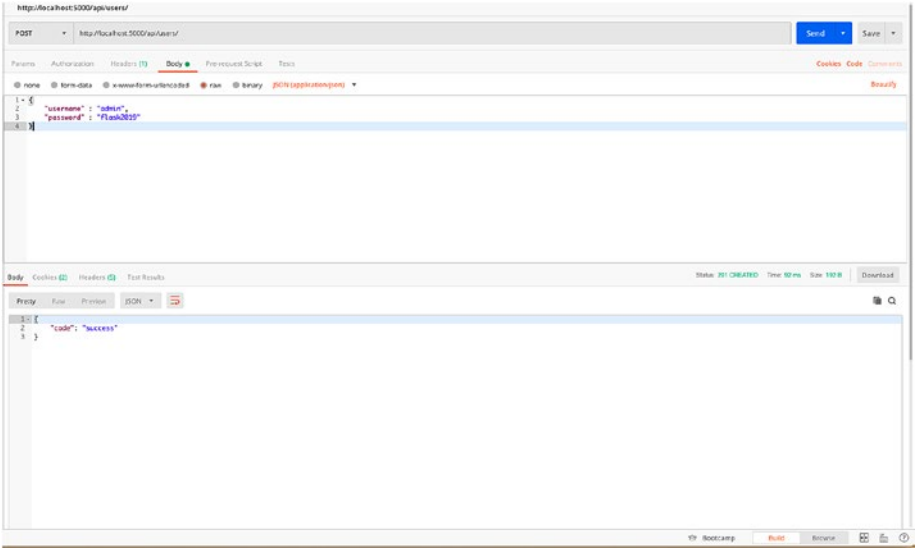

Figure 3-11. *User signup endpoint*

So our new user has been created (Figure 3-11); next we'll try logging in with the same credentials and get the JWT.

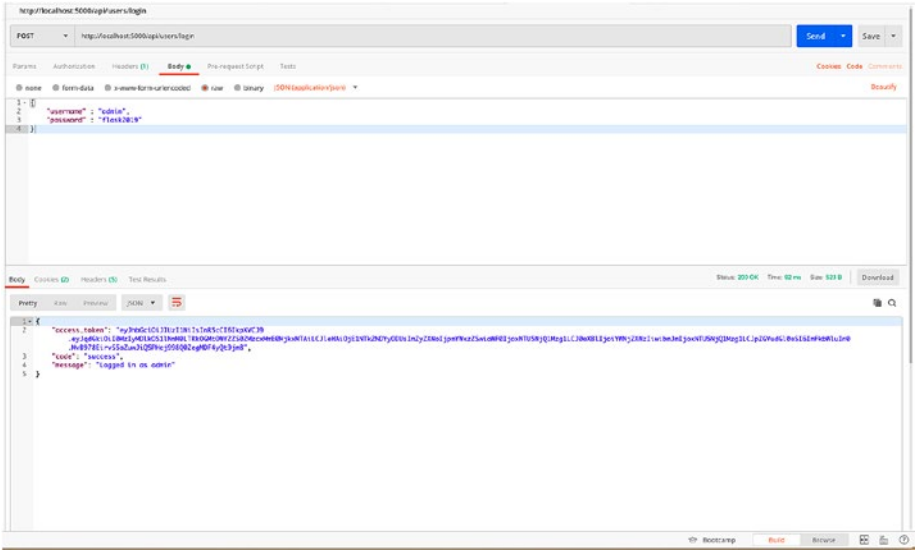

Figure 3-12. *User login endpoint*

As you see in Figure 3-12, we have successfully logged in using the newly created users. Now let's try accessing the POST author route to which we recently added jwt_required decorator (Figure 3-13).

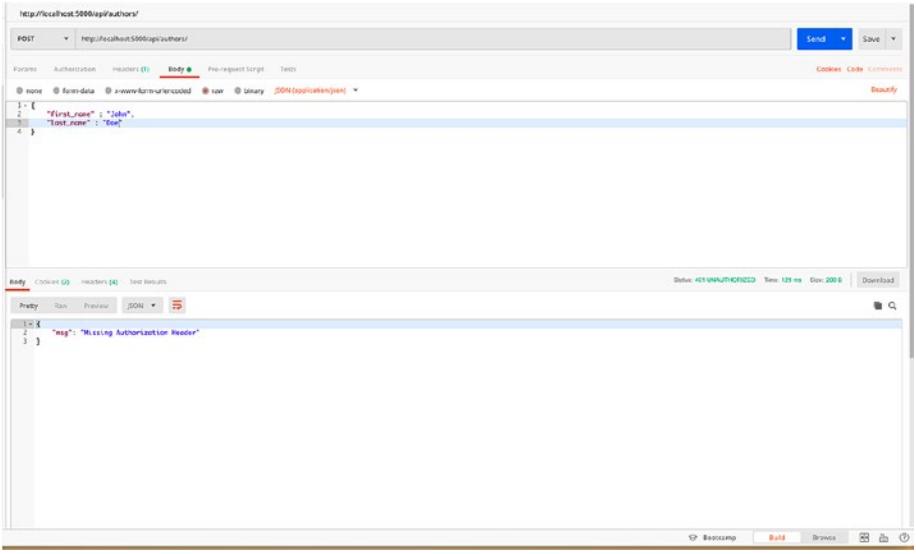

Figure 3-13. *POST author route without JWT Token*

As you see in Figure 3-14, we are not able to access the POST author route anymore, and the jwt_required decorator responded back with 401 error. Now let's try accessing the same route by supplying the JWT in header. In the header section of the request in Postman, add the token with a key called Authorization, and then in the value add Bearer <token> to supply the JWT Token like in Figure 3-14.